

MODULE 1

WSL and Linux Command-Line Fundamentals

01

TIME	8 hours · four 2-hour sessions
LABS	Lab 01 Linux & WSL · Lab 02 CLI file manipulation
PREREQUISITES	None: just WSL with Ubuntu installed

Module 1 objectives

M1 · Linux CLI

1 Explain Linux, WSL, terminal, shell and Bash

2 Navigate with absolute and relative paths

3 Reach Windows files from Linux via /mnt/c

4 Manage files: create, copy, move, delete, link

5 Inspect data files without opening them

6 Combine commands with pipes, redirection and chaining

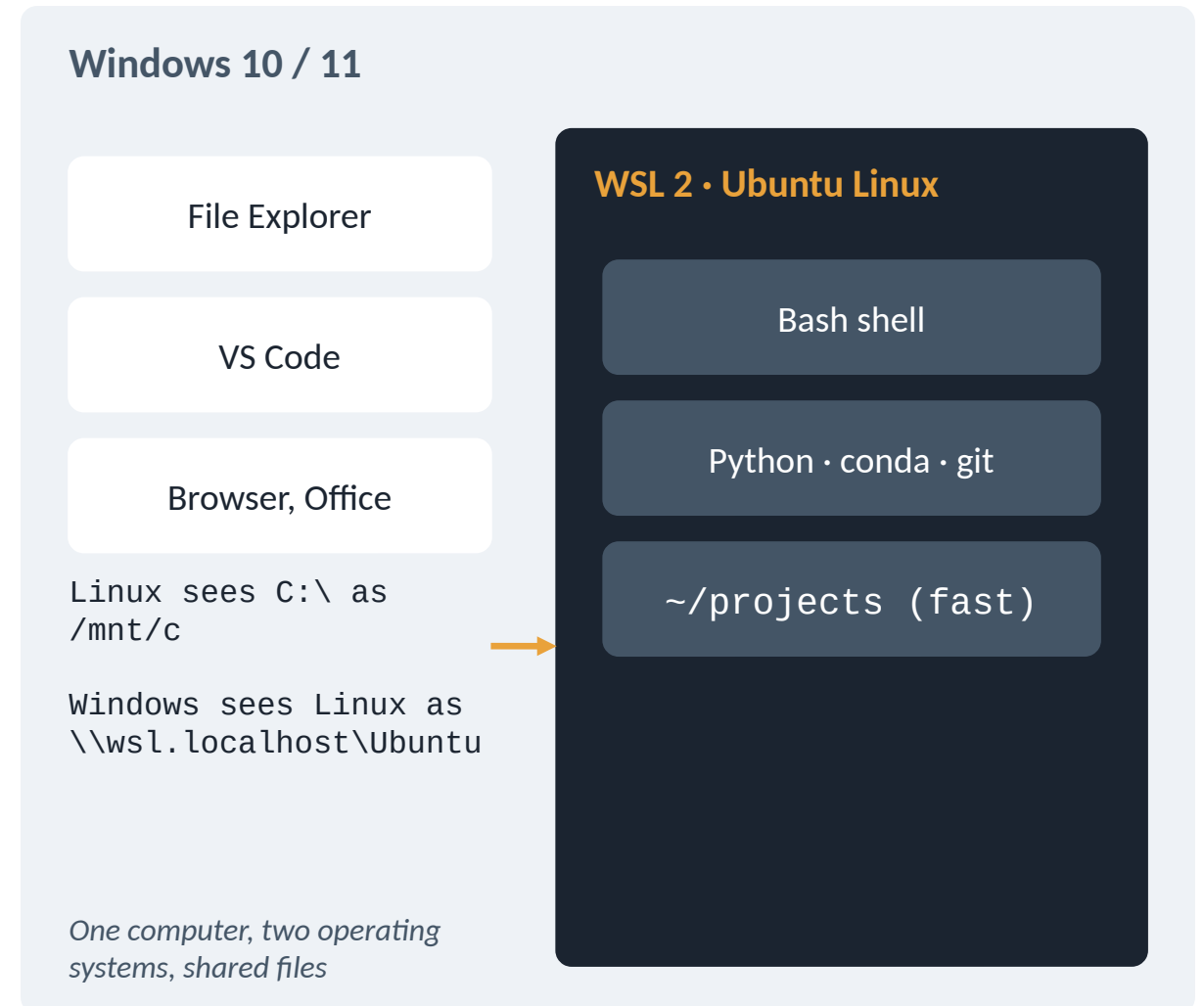
7 Use permissions and environment variables (\$PATH)

8 Compress and archive files

Linux and WSL

M1 • Linux CLI

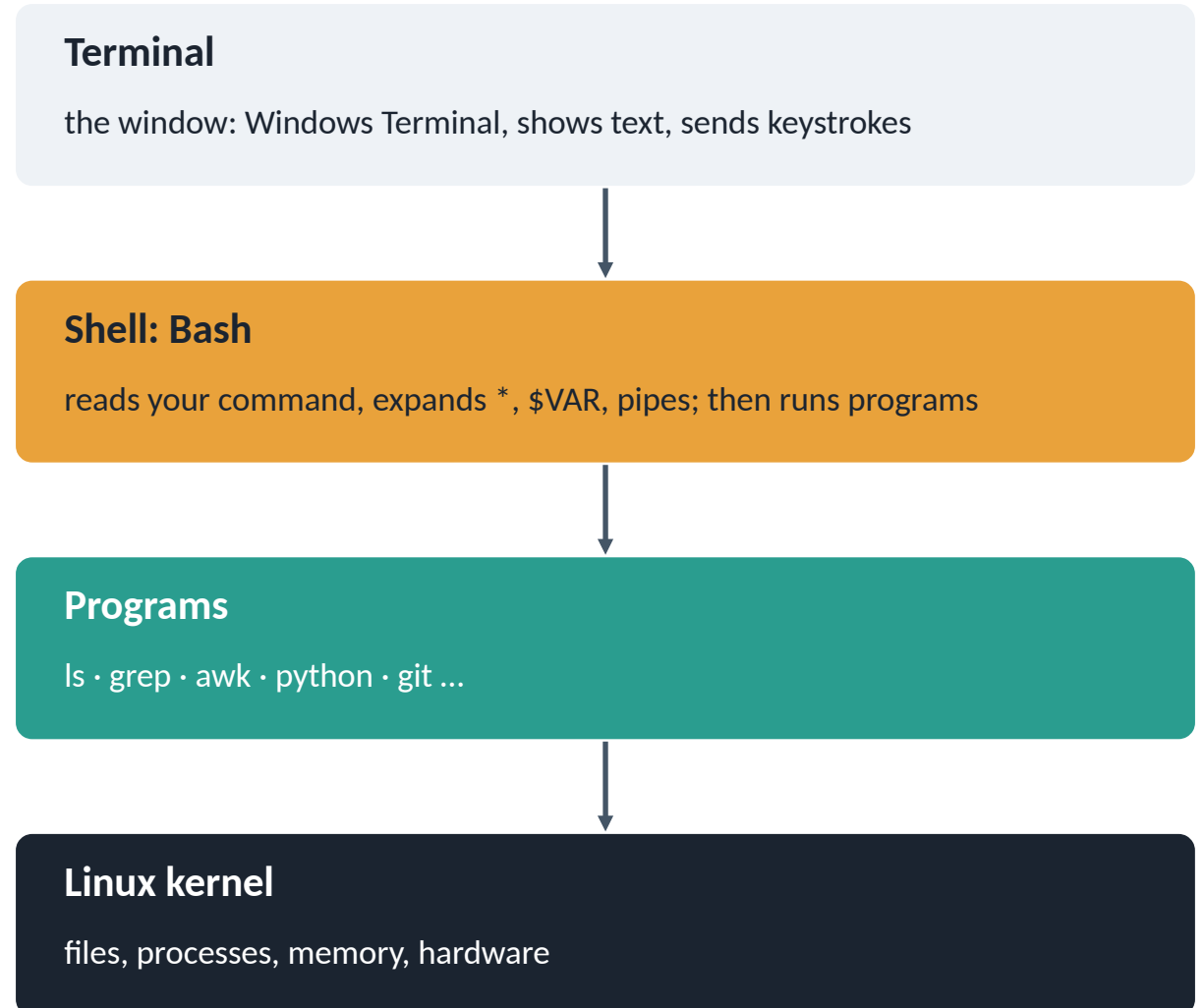
- **Linux** is an operating system, like Windows or macOS.
- It runs most servers, clusters and clouds, so scientific computing happens on Linux.
- **WSL** (Windows Subsystem for Linux) runs a real Ubuntu Linux inside Windows. No dual boot.
- Windows and Linux share files: Linux sees C:\ as /mnt/c.
- **Rule:** keep projects in your Linux home (~). It's faster and permissions work.



Terminal, shell and Bash

M1 · Linux CLI

- **Terminal:** the window that shows text.
- **Shell:** the program that interprets what you type.
- **Bash:** the most common shell, and the one on almost every server.
- Analogy: the terminal is a telephone, the shell is the person who answers, and Bash is the language they speak.
- Everything you type goes through the shell before a program runs.



CLI vs GUI

M1 · Linux CLI

Graphical interface (GUI)

- Easy to discover: menus and buttons
- Hard to repeat exactly
- Hard to automate or document
- Needs a screen: no servers or clusters
- One file at a time

Command line (CLI)

- Must learn the commands
- Every step is text: repeatable and shareable
- Easy to automate with scripts
- Works the same on laptops, servers, clusters
- Rename 1,000 files in one line

The CLI takes longer to learn and then saves time forever, because what you typed can be saved, rerun and shared.

Your first commands

M1 · Linux CLI

```
bash
ada@LAPTOP:~$ whoami
ada
$ pwd
/home/ada
$ ls
scripting-101
$ ls -la
drwxr-x--- 5 ada ada 4096 Oct  5 10:00 .
drwxr-xr-x 3 root root 4096 Oct  1 09:00 ..
-rw-r--r-- 1 ada ada 3771 Oct  1 09:00 .bashrc
drwxr-xr-x 9 ada ada 4096 Oct  5 10:00 scripting-101
$ date
Mon Oct  5 10:02:11 CEST 2026
```

user@host :~\$

the prompt: who, which machine, where (~ = home)

pwd

print working directory: where am I?

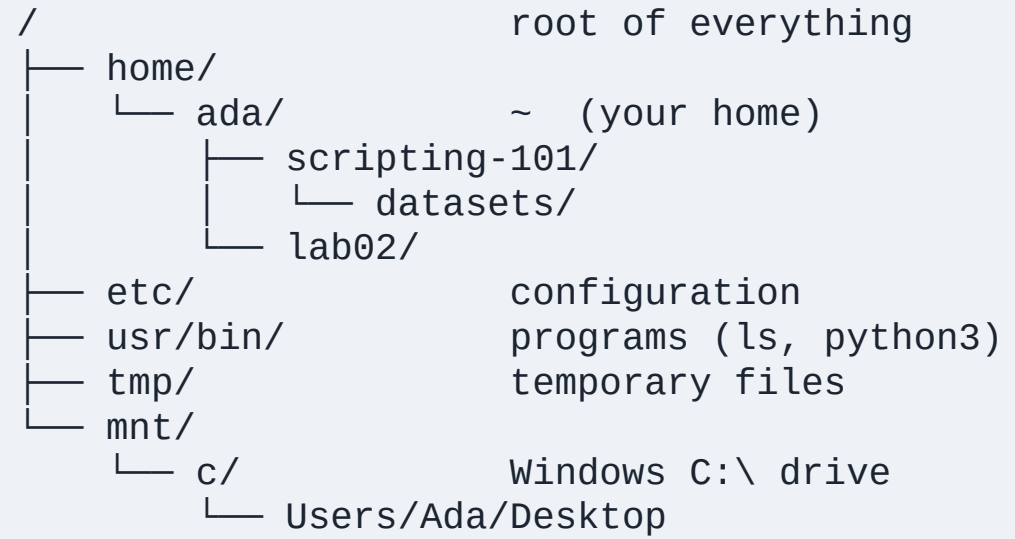
ls -la

list all files (incl. hidden .files), long format

Paths: addresses for files

- `/` is the root of everything.
- `~` is your home folder: `/home/<you>`.
- **Absolute path:** starts at `/` — a full postal address.
`/home/ada/scripting-101/datasets`
- **Relative path:** starts where you are — directions from here. `../datasets`
- `.` and `..` are this folder and the parent folder.
- **Tab** completes paths and prevents typos.

the Linux filesystem tree



Anatomy of a command

M1 · Linux CLI

ls

command:
what to run

-l

option:
long format

-a

option:
all files

-h

option:
human sizes

~/scripting-101

argument:
what to act on

bash

```
$ ls -lah ~/scripting-101
```

```
total 64K
```

```
drwxr-xr-x 9 ada ada 4.0K Oct  5 10:00 .
```

```
-rw-r--r-- 1 ada ada 3.1K Oct  5 10:00 README.md
```

```
drwxr-xr-x 7 ada ada 4.0K Oct  5 10:00 datasets
```

```
# short options combine: -l -a -h == -lah    long options:  --all  --human-readable
```

```
$ man ls      # the manual: q to quit, /word to search
```

Moving around

M1 • Linux CLI

```
bash
$ cd ~/scripting-101/datasets      # absolute (via ~)
$ pwd
/home/ada/scripting-101/datasets
$ cd field_trials                  # relative: a folder inside here
$ cd ..                           # up one level
$ cd ../../                       # up two levels
$ cd -                            # back to where you were
$ cd                              # home
$ mkdir -p ~/lab01/project/{data,scripts,results}
$ ls ~/lab01/project
data  results  scripts
$ ls /mnt/c/Users/Ada/Desktop      # Windows Desktop from Linux
$ explorer.exe .                  # open this Linux folder in Windows
```

Use Tab to complete names. If Tab doesn't complete, the path is wrong.

Managing files and folders

M1 • Linux CLI

Command	What it does	Example	Watch out
<code>mkdir -p</code>	make folders (and parents)	<code>mkdir -p data/raw</code>	-p = no error if it exists
<code>touch</code>	create an empty file	<code>touch notes.txt</code>	also updates the timestamp
<code>cp</code> / <code>cp -r</code>	copy a file / a folder	<code>cp -r weather backup/</code>	-r needed for folders
<code>mv</code>	move or rename	<code>mv a.csv data/raw/</code>	overwrites without asking
<code>rm</code> / <code>rm -r</code>	delete file / folder	<code>rm -i old.csv</code>	NO recycle bin: Is the pattern first
<code>rmdir</code>	remove an EMPTY folder	<code>rmdir tmp</code>	fails if not empty (safe)
<code>ln -s</code>	symbolic link (shortcut)	<code>ln -s ~/big_data data</code>	breaks if the target moves

Looking inside files without opening them

M1 • Linux CLI

Command	Shows	Example
<code>cat</code>	the whole file	<code>cat sites.csv</code>
<code>less -S</code>	page through (q quit, / search)	<code>less -S plot_yields.csv</code>
<code>head / tail</code>	first / last lines	<code>head -n 5 plot_yields.csv</code>
<code>wc -l</code>	number of lines	<code>wc -l plot_yields.csv → 1444</code>
<code>file</code>	file type, line endings	<code>file messy/*.csv</code>
<code>cat -A</code>	hidden characters (^M = Windows)	<code>cat -A f.csv head -2</code>
<code>du -sh / df -h</code>	folder size / free disk	<code>du -sh datasets/*</code>
<code>zcat f.gz head</code>	peek into compressed files	<code>zcat reads/S01.fastq.gz head -4</code>

Pipes: small tools, big answers

M1 • Linux CLI

cut -d, -f5

take column 5 (variety)

tail -n +2

drop the header line

sort

put equal values next to each other

uniq -c

count each distinct value

bash

```
$ cut -d, -f5 plot_yields.csv | tail -n +2 | sort | uniq -c
      2 AMBER      356 Amber      1 Amber      357 Bolt      1 Bolt
    361 Crest       1 Crest      1 DUNE      360 Dune      1 amber      2 bolt
# 11 spellings of 4 varieties: what would 'mean yield per variety' give now?
```

Standard streams and redirection

- Every program has three streams: input, output, errors.
- `>` sends output to a file (overwrite); `>>` appends.
- `2>` sends errors to a file.
- `> f 2>&1` sends both to the same file.
- `|` connects output to the next program's input.
- ⚠ `sort f > f` empties `f` before `sort` reads it.



```
$ ls data nope > out.txt      # error on screen
$ ls data nope > out.txt 2> err.txt
$ ls data nope > all.txt 2>&1 # both in file
```

Redirection and chaining cheat sheet

M1 • Linux CLI

Syntax	Meaning	Example
<code>cmd > out</code>	stdout to file (overwrite)	<code>cut -d, -f2 f.csv > sites.txt</code>
<code>cmd >> out</code>	append	<code>echo "# done \$(date)" >> log.txt</code>
<code>cmd < in</code>	read stdin from a file	<code>wc -l < f.csv</code>
<code>cmd 2> err</code>	stderr to file	<code>ls a nope 2> errors.txt</code>
<code>cmd > all 2>&1</code>	both into one file	<code>./run.sh > run.log 2>&1</code>
<code>cmd1 cmd2</code>	pipe	<code>grep AMES f.csv wc -l</code>
<code>cmd1 && cmd2</code>	cmd2 only if cmd1 succeeded	<code>mkdir out && cp f.csv out/</code>
<code>cmd1 cmd2</code>	cmd2 only if cmd1 failed	<code>ls nope echo "missing"</code>
<code>cmd1 ; cmd2</code>	always both	<code>date ; whoami</code>

Wildcards, grep and find

M1 · Linux CLI

```
bash
$ ls weather/weather_AMES_*.csv          # * = any characters
$ ls weather/weather_????_2023.csv | wc -l # ? = exactly one character
6
$ grep -c '^>' genomics/reference.fasta  # count FASTA headers
5
$ grep -i drought "messy/field notes 2023.txt" # quotes: spaces in name!
2023-07-19 MANH: drought stress visible, leaf rolling by 11:00
$ find sensors -name '*.csv' | wc -l
168
$ find sensors -type f -empty
sensors/LINC/2024-07-03/logger_02.csv
```

** ? [0-9] are expanded by the shell · grep finds lines (-c count, -i ignore case, -v invert) · find searches folders recursively*

Permissions: who may read, write, run

M1 • Linux CLI

- Every file has permissions for user, group and others.
- Scripts need the execute (x) bit to run as ./script.sh.
- Protect raw data by removing write permission.
- `ls -l` shows them; `chmod` changes them.
- Under `/mnt/c` permissions are only emulated: another reason to work in `~`.



letter	on a file	value
r	read	4
w	write / change	2
x	execute (run)	1

```
chmod u+x run.sh      # user may execute
chmod 755 run.sh      # rwx r-x r-x
chmod a-w raw.csv     # read-only for all
```

Copy vs symbolic link

M1 • Linux CLI

`cp big.csv copy.csv`

- An independent duplicate
- Uses the disk space again
- Editing the original does not change the copy
- Survives deletion of the original
- Use for backups and snapshots

`ln -s big.csv link.csv`

- A signpost pointing at the original
- Uses (almost) no space
- Always shows the current original
- Breaks if the original moves or is deleted
- Use to share large datasets between projects

Large datasets: link, don't copy. Results you want to freeze: copy.

Compression and archives

M1 • Linux CLI

```
bash
$ ls -lh plot_yields.csv
-rw-r--r-- 1 ada ada 72K plot_yields.csv
$ gzip plot_yields.csv && ls -lh plot_yields.csv.gz
-rw-r--r-- 1 ada ada 13K plot_yields.csv.gz          # 5.5x smaller
$ zcat plot_yields.csv.gz | head -2                 # read without unpacking
$ gunzip plot_yields.csv.gz
$ tar -czvf weather_backup.tar.gz weather/          # c create, z gzip, v verbose, f file
$ tar -tzf weather_backup.tar.gz | head -3          # t list
$ tar -xzf weather_backup.tar.gz -C restore/        # x extract
$ zip -r weather.zip weather/ && unzip -l weather.zip
```

Text data compress 4–10×. Keep big files compressed and read them with zcat.

Environment variables and \$PATH

M1 · Linux CLI

```
bash
$ echo $HOME $USER
/home/ada ada
$ echo $PATH | tr ':' '\n'
/home/ada/miniforge3/envs/scripting101/bin
/usr/local/bin
/usr/bin
/bin
$ which python
/home/ada/miniforge3/envs/scripting101/bin/python
$ greeting="Hello"          # no spaces around =
$ echo "$greeting, $USER"
Hello, ada
$ export COURSE=~/.scripting-101 # for child programs
```

\$PATH

folders searched, in order, when you type a command name

which

shows which program file will actually run

export

passes a variable on to programs you start

Module 1: common mistakes

M1 · Linux CLI

SYMPTOM	CAUSE	FIX
No such file or directory	wrong folder, typo, Windows path	<code>pwd · ls · Tab · /mnt/c/...</code>
cd: too many arguments	spaces in the name not quoted	<code>cd "field notes"</code>
Permission denied	no execute bit / system folder	<code>chmod u+x run.sh · work in ~</code>
file is suddenly empty	sort f > f truncates f first	<code>sort f > f.sorted</code>
uniq -c counts look wrong	input not sorted	<code>sort uniq -c</code>
^M at line ends	Windows CRLF line endings	<code>dos2unix f · tr -d '\r'</code>

Module 1: best practices

M1 · Linux CLI



Live in ~

Projects in the Linux home folder; /mnt/c only to move files in and out.



Use Tab and ↑

Tab-complete everything; reuse commands from history (Ctrl+R).



Look before you delete

Run ls with the same pattern before rm; there is no undo.



Build pipelines stepwise

Add one command at a time and check the output each time.



Quote names

Quote paths with spaces, or better, never put spaces in file names.



Keep data compressed

Read .gz files directly with zcat; don't unpack big files.

Lab 01: find your way around

M1 • Linux CLI

HANDS-ON

Lab 01 • Linux & WSL • 90 min

1. Read the prompt; run `whoami`, `hostname`, `date`, `uname -a`
2. Explore `/`, `/home`, `/etc`, `/usr/bin`, `/tmp`, `/mnt` and describe each
3. Create `~/lab01/project/{data,scripts,results}` and move around with relative paths
4. Reach your Windows Desktop through `/mnt/c`; open `~` in Explorer
5. Use `man`, `--help`, `which`, `type`, `file` and `history`
6. Inspect `$HOME`, `$USER` and `$PATH`; create and export a variable

Expected output

```
bash
$ pwd
/home/ada
$ cd ~/lab01/project/results
$ cd ../data && pwd
/home/ada/lab01/project/data
$ type cd
cd is a shell builtin
$ which python3
/usr/bin/python3
```

Lab 02: question the data with pipelines

M1 • Linux CLI

HANDS-ON

Lab 02 • CLI file manipulation • 150 min

1. How many plots are in plot_yields.csv? Which column is the yield?
2. How many plots per site? Why do three sites have 241?
3. How many yields are missing (NA)? Why is grep -c NA unsafe?
4. What are the 5 highest yields: are they believable?
5. Find the empty and header-only sensor files with find
6. Archive weather/ with tar; compare the sizes
7. Make a symlink to the datasets and explain copy vs link

Expected output

```
bash
$ tail -n +2 plot_yields.csv | wc -l
1443
$ cut -d, -f8 plot_yields.csv \
> | grep -cx NA
25
$ tail -n +2 plot_yields.csv \
> | sort -t, -k8,8gr | head -1
STPL-2023,STPL,2023,140,Bolt,
180,3,99.9,15.7,215,
$ tail -n +2 plot_yields.csv \
> | sort | uniq -d | wc -l
3
```

Module 1: check your understanding

M1 · Linux CLI

Q1

You are in `~/lab02/data`. Give the absolute and the relative path to `~/lab02/scripts`.

Q2

Why must `sort` come before `uniq -c`?

Q3

What exactly does `./run.sh > run.log 2>&1` do?

Q4

What is the difference between `chmod 755` and `chmod u+x`?

Q5

Your colleague deleted the folder your symlink points to. What happens?

Q6

Write one pipeline: the three site-years with the most missing yields.

Module 1 summary

M1 · Linux CLI

- ✓ Terminal = window, shell = interpreter, Bash = the shell we use
- ✓ Paths: / root, ~ home, . here, .. parent; absolute vs relative
- ✓ Files: mkdir, cp, mv, rm, ln -s; look with head, tail, less, wc
- ✓ Pipes and redirection turn small tools into data questions
- ✓ Permissions (chmod), \$PATH, compression (gzip, tar)
- ✓ Work in ~, use Tab, look before you delete

NEXT

Module 2: save commands in files: Bash scripts.